

Aula 5

Redes Neurais Artificiais e Machine Learning

Prof. Luciano Frontino de Medeiros

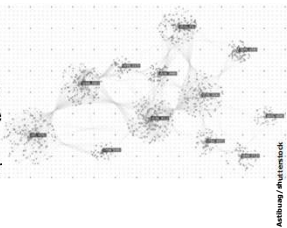
1

Algoritmos de Aglomeração (Clustering e DBSCAN)

2

Algoritmos de aglomeração

- Tipo de algoritmo genérico usado para designar um amplo espectro de métodos numéricos de análise de dados com o objetivo de descobrir grupos homogêneos de objetos



3

- A tarefa de agrupamento de objetos em diferentes grupos pode permitir a representação de maneira conveniente bases de dados muito grandes, visando facilitar a compreensão
- Estes algoritmos são caracterizados pela aprendizagem de maneira não supervisionada



4

Indicadores

- Coesão interna ou homogeneidade
 - Permite demonstrar o quanto os objetos que pertencem a um grupo compartilham das mesmas características
- Isolamento externo
 - Permite verificar o quanto os elementos de um grupo são separados dos elementos de outro grupo

5

Vetor de características

- Vetor que contém os atributos relativos aos objetos que estão sendo analisados (dimensões)
- Os atributos variam em uma faixa de valores, conforme as suas propriedades dentro de um domínio sendo abordado
- Por exemplo, idade, altura, largura, quantidade vendida

6

Iris Dataset

Visualização dos primeiros registros (Jupyter Notebook)

	sepal-length	sepal-width	petal-length	petal-width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Utilização

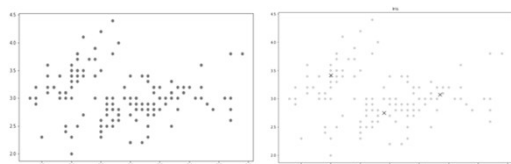
- Marketing: segmentação de clientes e de produtos
- Agricultura: categorização de plantas, solos e frutos
- Meteorologia: identificação de padrões climáticos
- Finanças: identificação de perfil de clientes
- Medicina: categorização de pacientes, diagnósticos e remédios

Algoritmos de agrupamento

- Baseado em protótipo: K-means e k-medoids (Amaral, 2016, p. 109)
- Baseado em densidade: DBSCAN (Amaral, 2016, p. 110)

- Clustering Hierárquico (Single linkage) (Amaral, 2016, p. 111)
- Árvore geradora mínima (MST – minimum spanning tree) (Castro; Ferrari, 2016, p. 127)
- Fuzzy k-means (Duda; Hart; Stork, 2001, p. 14)

Comprimento da sépala x largura da sépala



Construção do Modelo em Python e Jupyter Notebook (Iris)

Iris Dataset

- Publicado por Fischer em 1936
- Contém 150 instâncias com 4 atributos diferentes
- Está pré-classificado, dividindo as instâncias em três rótulos diferentes
 - Setosa, Versicolour e Virginica



nadezhda F/shutterstock

Importação de bibliotecas

- Bibliotecas Pandas e Matplotlib serão importadas, junto com o módulo do KMeans que é parte da Scikit-learn

```
In [1]: 1 import pandas as pd
        2 import matplotlib.pyplot as plt

In [2]: 1 from sklearn.cluster import KMeans
```

Leitura do Dataset

- Pode ser feita no próprio site do repositório onde se encontra o dataset, ou pode ser lido a partir dos dados instalados localmente

```
In [2]: data = pd.read_csv("https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data", header=None)
        data.columns = ["sepal-length", "sepal-width", "petal-length", "petal-width", "class"]
        del data["class"]
        data.head()
```

```
Out[2]:
```

	sepal-length	sepal-width	petal-length	petal-width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Estatística descritiva

- Describe() - comando para exibir a estatística descritiva (Pandas)

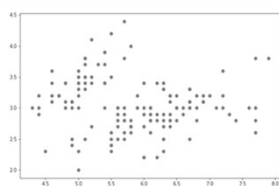
```
In [4]: 1 # Fazendo a estatística descritiva
        2 data.describe()

Out[4]:
```

	sepal-length	sepal-width	petal-length	petal-width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828068	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Visualização

```
In [5]: 1 # Gerando o gráfico, mostrando apenas o relacionamento entre o comprimento e largura da sépala
        2 fig = plt.figure(figsize=(10,6))
        3 sp = fig.add_subplot(1,1,1)
        4 sp.scatter(data['sepal-length'], data['sepal-width'])
```



Prática do Modelo (Iris)

Criação do Objeto Kmeans()

- **n_clusters:** número de clusters
- **init:** algoritmo que será executado para a criação dos centros comuns ('kmeans++', 'random')
- **n_init:** número de vezes que o algoritmo irá iniciar aleatoriamente para pegar a melhor configuração inicial

19

- **max_iter:** número máximo de iterações que o algoritmo será executado
- **tol:** tolerância entre duas iterações para estabelecer como critério de parada do algoritmo
- **random_state:** número aleatório utilizado como semente para o algoritmo de geração randômica dos centros

20

Objeto KMeans()

- Três clusters, o algoritmo de inicialização dos centros comuns é o 'k-means++', o número de execuções aleatórias para pegar a melhor configuração inicial será de 10, e um número aleatório qualquer

```
In [5]: kmeans = KMeans(n_clusters=3, init='k-means++', n_init=10, random_state=1234)
kmeans
Out[5]: KMeans(n_clusters=3, random_state=1234)
```

21

Predição das instâncias

```
In [6]: data["clusters"] = kmeans.fit_predict(data)
```

```
Out[6]:
```

	sepal-length	sepal-width	petal-length	petal-width	clusters
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	1
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	1

150 rows x 5 columns

22

Agrupamento

- **groupby()** - agrupa as instâncias e calcula a quantidade de cada classe atribuída (Pandas)

```
In [7]: group_count = data.groupby("clusters").size().reset_index(name='counts')
group_count
```

```
Out[7]:
```

clusters	counts
0	50
1	62
2	38

23

Centros comuns

```
In [8]: group = data.groupby("clusters")
group
```

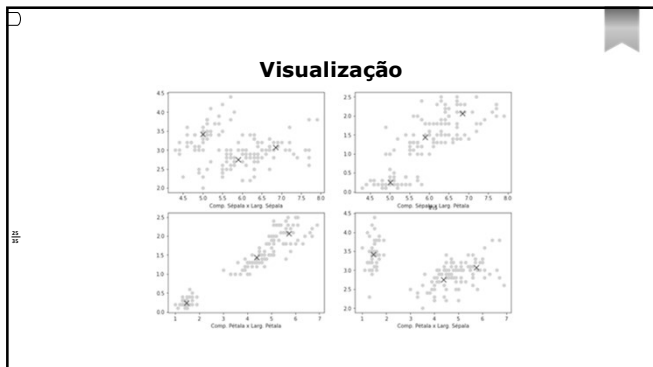
```
Out[8]: <pandas.core.groupby.generic.DataFrameGroupBy object at 0x0000027A2DDC6FD0>
```

```
In [9]: gm = group.mean()
gm
```

```
Out[9]:
```

	sepal-length	sepal-width	petal-length	petal-width
clusters				
0	5.006000	3.418000	1.464000	0.244000
1	5.901613	2.748387	4.393548	1.433871
2	6.850000	3.073684	5.742105	2.071053

24



25

Construção do Modelo em Python e Jupyter Notebook (Wholesale Customer)

26

- ### "Wholesale Customer" (cliente de atacado)
- Contém 440 instâncias que são clientes, com o valor total de vendas anuais em cada um dos 6 atributos
 - Alimentos frescos ("fresh"),
 - Derivados de leite ("milk"),
 - Gêneros alimentícios ("grocery"),
 - Congelados ("frozen"),
 - Higiene e limpeza ("detergents & paper"),
 - Doces ("delicatessen")

27

Objetivo

- Identificar grupos de clientes com perfis de compra similares quanto aos valores comprados
- Para facilitar a análise, dois campos serão excluídos neste momento, canal e região

```
In [21]: data = pd.read_csv("https://archive.ics.uci.edu/ml/machine-learning-databases/w00292/wholesale20customers20data.csv")
del data["channel"]
del data["region"]
data.head()
```

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicatessen
0	12060	9658	7501	214	2014	1338
1	1057	9910	9558	1762	3280	1776
2	6353	8808	7884	2405	3516	7844
3	13205	1196	4221	8454	507	1788
4	22815	8410	7198	3815	1777	8185

28

Estatística descritiva

```
In [22]: # Fazendo-se a estatística descritiva
data.describe()
```

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicatessen
count	440.000000	440.000000	440.000000	440.000000	440.000000	440.000000
mean	12060.297727	5796.265909	7951.277273	3071.931818	2881.493182	1524.870455
std	12847.328865	7380.377175	9503.162829	4854.673333	4767.854448	2820.105937
min	3.000000	55.000000	3.000000	25.000000	3.000000	3.000000
25%	3127.750000	1533.000000	2153.000000	742.250000	256.750000	408.250000
50%	8504.000000	3627.000000	4755.500000	1526.000000	816.500000	965.500000
75%	16933.750000	7190.250000	10655.750000	3554.250000	3922.000000	1820.250000
max	112151.000000	73498.000000	92780.000000	60869.000000	40827.000000	47943.000000

29

Criação do objeto Kmeans()

```
In [23]: # Atribuindo-se os valores a outro data frame
data_array = data.values

In [24]: # Cria-se o objeto KMeans com 5 clusters
kmeans = KMeans(n_clusters=5, init='k-means++', n_init=10, random_state=1234)
kmeans

Out[24]: KMeans(algorithm='auto', copy_x=True, init='k-means++', max_iter=300,
n_clusters=5, n_init=10, n_jobs=1, precompute_distances='auto',
random_state=1234, tol=0.0001, verbose=0)
```

30

Prática do Modelo (Whole Sale Customer)

Predição

```
In [25]: # Será-se uma nova coluna no data frame
data["clusters"] = kmeans.fit_predict(data_array)
data
```

```
Out[25]:
```

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassas	clusters
0	12969	9656	7561	214	2674	1338	0
1	7027	9810	9568	1762	3293	1776	0
2	8383	8808	7884	2405	3516	7944	0
3	12261	1198	4231	6404	507	1518	0
4	22615	5410	7198	3016	1777	5105	3
5	8413	8209	5126	956	1766	1451	0
6	12126	3189	6075	480	3140	545	0
7	7379	4966	9426	1969	3321	2588	0
8	5063	3648	6192	425	1716	750	0
9	8006	1108	18881	1159	7425	2088	1
10	1086	5402	12074	4492	5677	1514	1
11	13146	9124	4323	1420	549	487	0
12	31714	12319	11757	287	3681	2631	3
13	21217	6208	14982	3096	6707	602	3

Ordenação por clusters

```
In [26]: # Ordenando-se pelas clusters
data.sort_values(by="clusters")
```

```
Out[26]:
```

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassas	clusters
0	12969	9656	7561	214	2674	1338	0
252	6623	1860	4740	7883	205	1693	0
250	3191	1993	1799	1730	234	710	0
249	8040	3795	2070	6340	918	291	0
247	13569	346	489	2077	44	659	0
246	8885	2428	1777	1777	430	610	0
244	6202	7775	10817	1183	3143	1970	0
243	11210	3576	5119	561	1682	2398	0
238	7363	475	585	1112	72	216	0
236	8635	820	3047	2312	415	225	0
235	1838	6380	2824	1218	1216	295	0
233	964	4984	3316	937	409	7	0

Agrupamento

```
In [27]: # Utilizando-se GroupBy do Pandas
group = data.groupby("clusters")
```

```
In [28]: # Contabilizando-se itens por cluster
group_count = data.groupby("clusters").size().reset_index(name='counts')
group_count
```

```
Out[28]:
```

	clusters	counts
0	0	226
1	1	81
2	2	21
3	3	105
4	4	7

Visualização

```
In [30]: # Gerando-se a gráfico, mostrando as clusters com conjuntos de barras, cada uma representando uma característica
group.mean().add(["label","sum"], fillcolor="r", title="Consumo por Cluster")
```

```
Out[30]: output110.axes._subplots.AxesSubplot at 0x1247162988
```

